

TUGAS PRAKTIKUM
REKAYASA PERANGKAT LUNAK



Pertemuan 11 – Version Control dan Kolaborasi Tim

Nama kelompok :

- 1. Rahmawati Fauziah (2403015103)**
- 2. Putri aulia amanda (2403015048)**
- 3. Layla khairanni (2403015100)**
- 4. Muthia mua'adzarah (2403015039)**
- 5. Marlo Fatur Riziyy (2403015071)**

PROGRAM STUDI INFORMATIKA (SI)
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH PROF. DR. HAMKA
JAKARTA 2026

Nama: Rahmawati Fauziah

NIM : 2403015103

Kelas : 4b

Pertemuan 11 – Version Control dan Kolaborasi Tim

1. Tujuan

Pertemuan ini bertujuan agar mahasiswa mampu mengelola perubahan kode secara terstruktur menggunakan sistem version control Git, serta mampu berkolaborasi dalam tim pengembangan secara efektif melalui platform GitHub. Mahasiswa diharapkan memahami alur kerja branching, commit, dan merge yang merupakan praktik standar dalam pengembangan perangkat lunak profesional.

2. Teori Singkat

Version control adalah sistem yang mencatat setiap perubahan pada kode program sehingga tim dapat melacak riwayat perubahan, kembali ke versi sebelumnya bila terjadi kesalahan, serta bekerja secara paralel tanpa saling menimpa pekerjaan satu sama lain. Git adalah sistem version control terdistribusi yang paling banyak digunakan saat ini karena ringan, cepat, dan mendukung kolaborasi tim dengan baik.

Tanpa version control, pengembangan tim sangat rentan terhadap konflik kode. Bayangkan dua anggota tim mengedit file yang sama secara bersamaan lalu saling menimpa perubahan masing-masing. Git menyelesaikan masalah ini dengan sistem branching yang memungkinkan setiap fitur dikerjakan di cabang terpisah, kemudian digabungkan ke cabang utama melalui proses merge yang terkontrol.

3. Penggunaan Git pada Proyek tukang.in

3.1 Inisialisasi Repository

Repository proyek tukang.in dibuat di GitHub dengan nama 'tukang-in'. Setiap anggota tim melakukan clone repository ke mesin lokal masing-masing dan mengonfigurasi identitas Git berupa nama dan email agar setiap commit dapat dilacak ke penulisnya. Branch utama yang digunakan adalah main, yang hanya berisi kode yang sudah diuji dan siap produksi.

3.2 Strategi Branching

Setiap fitur baru dikerjakan di branch terpisah dengan penamaan yang deskriptif, misalnya feature/auth-login untuk fitur login, feature/pemesanan untuk fitur pemesanan, dan fix/validasi-form untuk perbaikan bug pada form. Penamaan ini memudahkan seluruh anggota tim memahami tujuan setiap branch tanpa harus membuka kodenya terlebih dahulu.

Branch fitur dibuat dari branch main yang terbaru untuk memastikan tidak ada konflik dengan kode yang sudah ada. Setelah fitur selesai dan diuji secara lokal, branch tersebut di-push ke GitHub dan dibuat pull request untuk direview oleh anggota tim lain sebelum digabung ke main. Proses review ini penting untuk menjaga kualitas kode dan memastikan tidak ada bug yang lolos ke kode utama.

3.3 Konvensi Commit Message

Setiap commit ditulis dengan pesan yang jelas dan deskriptif mengikuti format konvensional: tipe commit diikuti deskripsi singkat dalam bahasa Indonesia. Tipe yang digunakan antara lain feat untuk penambahan fitur baru, fix untuk perbaikan bug, docs untuk perubahan dokumentasi, dan refactor untuk perbaikan struktur kode tanpa mengubah fungsionalitas. Contoh pesan commit yang baik adalah 'feat: tambah fitur login pengguna dengan validasi email' atau 'fix: perbaiki error saat submit form pemesanan kosong'.

Commit yang baik bersifat atomik, artinya setiap commit hanya mencakup satu perubahan logis. Ini memudahkan proses pelacakan bug karena jika terjadi masalah, tim dapat dengan mudah mengidentifikasi commit mana yang menyebabkan masalah dan melakukan revert tanpa kehilangan perubahan lain.

4. Kolaborasi Tim

Kolaborasi tim pada proyek tukang.in dikelola melalui GitHub dengan memanfaatkan fitur Issues untuk mencatat tugas dan bug, serta Pull Request untuk proses code review. Setiap anggota tim bertanggung jawab atas satu atau beberapa fitur yang ditetapkan di awal sprint. Pembagian ini didokumentasikan di GitHub Projects agar semua anggota tim dapat memantau progress pengerjaan secara transparan.

Ketika terjadi merge conflict, penyelesaian dilakukan secara bersama dengan mendiskusikan bagian kode yang berkonflik dan memutuskan versi mana yang akan dipertahankan. Setiap konflik yang diselesaikan selalu dikomunikasikan kepada seluruh tim agar tidak ada anggota yang bekerja berdasarkan kode yang sudah usang.